

Chapter 2 Assignment

Student: UWIZEYE Docile

Student ID: 27549

Course: Introduction to Big Data with Python

Assignment: Chapter 2 - Control Flow and Functions

Task 1: Smart Grade Classifier (5 marks)

```
In [1]: # Task 1: Smart Grade Classifier
# Function returns both Letter grade and remark

def get_grade(score):
    """Return letter grade and remark based on score."""
    if score >= 90:
        return 'A', 'Distinction'
    elif score >= 80:
        return 'B', 'Merit'
    elif score >= 70:
        return 'C', 'Credit'
    elif score >= 60:
        return 'D', 'Pass'
    else:
        return 'F', 'Fail'

# List of at least 10 students
students = [
    ('Alice Uwase', 92),
    ('Bob Mugisha', 78),
    ('Carol Ingabire', 85),
    ('David Habimana', 45),
    ('Eve Mutoni', 63),
    ('Frank Nkurunziza', 91),
    ('Grace Umutoni', 72),
    ('Henri Gasabo', 55),
    ('Iris Kaneza', 88),
    ('Jean Bizimana', 67),
]

# Track grade counts and scores
grade_counts = {'A': 0, 'B': 0, 'C': 0, 'D': 0, 'F': 0}
all_scores = []
pass_count = 0

print('=' * 55)
print(f"{'NAME':<22} {'SCORE':>6} {'GRADE':>6} {'REMARK':>12}")
print('=' * 55)

for name, score in students:
    grade, remark = get_grade(score)
    grade_counts[grade] += 1
    all_scores.append(score)
    if grade != 'F':
```

```

        pass_count += 1
    print(f'{name:<22} {score:>6} {grade:>6} {remark:>12}')

print('=' * 55)
print(f'Total students : {len(students)}')
for g, count in grade_counts.items():
    print(f'Grade {g} : {count}')
print(f'Highest score : {max(all_scores)}')
print(f'Lowest score : {min(all_scores)}')
print(f'Class average : {sum(all_scores)/len(all_scores):.2f}')
print(f'Pass rate : {pass_count/len(students)*100:.1f}%')
print('=' * 55)

```

```

=====
NAME                SCORE  GRADE  REMARK
=====
Alice Uwase         92    A    Distinction
Bob Mugisha         78    C    Credit
Carol Ingabire      85    B    Merit
David Habimana      45    F    Fail
Eve Mutoni          63    D    Pass
Frank Nkurunziza    91    A    Distinction
Grace Umutoni       72    C    Credit
Henri Gasabo        55    F    Fail
Iris Kaneza         88    B    Merit
Jean Bizimana       67    D    Pass
=====
Total students : 10
Grade A : 2
Grade B : 2
Grade C : 2
Grade D : 2
Grade F : 2
Highest score : 92
Lowest score : 45
Class average : 73.60
Pass rate : 80.0%
=====

```

Task 2: Number Analysis Tool (5 marks)

In [2]: *# Task 2: Number Analysis Tool*

```

def full_number_analysis(numbers):
    """Perform comprehensive analysis on a list of numbers."""
    count = 0
    total = 0
    positives = 0
    negatives = 0
    zeros = 0
    evens = 0
    odds = 0
    minimum = numbers[0]
    maximum = numbers[0]

    # Loop to calculate all stats
    for n in numbers:
        count += 1
        total += n

```

```

        if n > 0: positives += 1
        elif n < 0: negatives += 1
        else: zeros += 1
        if n % 2 == 0: evens += 1
        else: odds += 1
        if n < minimum: minimum = n
        if n > maximum: maximum = n

    average = total / count
    above_avg = sum(1 for n in numbers if n > average)

    return count, total, average, minimum, maximum, positives, negatives, zeros,

# List of 15 numbers
numbers = [12, -5, 0, 34, -8, 7, 22, -1, 45, 0, 18, -3, 9, 30, -15]

count, total, avg, mn, mx, pos, neg, zer, evn, odd, abv = full_number_analysis(n

print('=' * 40)
print('          NUMBER ANALYSIS RESULTS')
print('=' * 40)
print(f' Numbers      : {numbers}')
print(f' Count        : {count}')
print(f' Sum           : {total}')
print(f' Average       : {avg:.2f}')
print(f' Minimum       : {mn}')
print(f' Maximum       : {mx}')
print(f' Positives     : {pos}')
print(f' Negatives     : {neg}')
print(f' Zeros         : {zer}')
print(f' Even          : {evn}')
print(f' Odd           : {odd}')
print(f' Above Avg     : {abv}')
print('=' * 40)

# List comprehensions for filtered versions
only_positive = [n for n in numbers if n > 0]
only_even     = [n for n in numbers if n % 2 == 0]
above_average = [n for n in numbers if n > avg]

print(f' Positive only : {only_positive}')
print(f' Even only     : {only_even}')
print(f' Above average : {above_average}')
print('=' * 40)

```

```
=====
NUMBER ANALYSIS RESULTS
=====
Numbers      : [12, -5, 0, 34, -8, 7, 22, -1, 45, 0, 18, -3, 9, 30, -15]
Count        : 15
Sum          : 145
Average      : 9.67
Minimum      : -15
Maximum      : 45
Positives    : 8
Negatives    : 5
Zeros        : 2
Even         : 8
Odd          : 7
Above Avg    : 6
=====
Positive only : [12, 34, 7, 22, 45, 18, 9, 30]
Even only     : [12, 0, 34, -8, 22, 0, 18, 30]
Above average : [12, 34, 22, 45, 18, 30]
=====
```

Task 3: Pattern Generator (5 marks)

In [3]: *# Task 3: Pattern Generator*

```
def right_triangle(n):
    """Print right-angled triangle of asterisks."""
    for i in range(1, n + 1):
        print('*' * i)

def inverted_triangle(n):
    """Print inverted triangle of asterisks."""
    for i in range(n, 0, -1):
        print('*' * i)

def pyramid(n):
    """Print centred pyramid of asterisks."""
    for i in range(1, n + 1):
        print(' ' * (n - i) + '*' * (2 * i - 1))

def hollow_rectangle(n):
    """Print hollow rectangle of asterisks."""
    height = n // 2
    for i in range(height):
        if i == 0 or i == height - 1:
            print('*' * n)
        else:
            print('*' + ' ' * (n - 2) + '*')

def number_triangle(n):
    """Print number triangle."""
    for i in range(1, n + 1):
        print(' '.join(str(j) for j in range(1, i + 1)))

# Call each function with n=5 and n=7
for n in [5, 7]:
    print(f'\n--- Right Triangle (n={n}) ---')
    right_triangle(n)
    print(f'\n--- Inverted Triangle (n={n}) ---')
```

```
inverted_triangle(n)
print(f'\n--- Pyramid (n={n}) ---')
pyramid(n)
print(f'\n--- Hollow Rectangle (n={n}) ---')
hollow_rectangle(n)
print(f'\n--- Number Triangle (n={n}) ---')
number_triangle(n)
```

--- Right Triangle (n=5) ---

```
*
**
***
****
*****
```

--- Inverted Triangle (n=5) ---

```
*****
****
***
**
*
```

--- Pyramid (n=5) ---

```
  *
  **
 ***
****
*****
*****
```

--- Hollow Rectangle (n=5) ---

```
*****
*****
```

--- Number Triangle (n=5) ---

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

--- Right Triangle (n=7) ---

```
*
**
***
****
*****
*****
*****
```

--- Inverted Triangle (n=7) ---

```
*****
*****
*****
****
***
**
*
```

--- Pyramid (n=7) ---

```
  *
  **
 ***
****
*****
*****
*****
*****
*****
*****
```

--- Hollow Rectangle (n=7) ---

```
*****
*      *
*****
```

```
--- Number Triangle (n=7) ---
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
```

Task 4: Mini Banking System (5 marks)

In [4]: *# Task 4: Mini Banking System*

```
def deposit(balance, amount):
    """Return new balance after deposit."""
    if amount <= 0:
        print(' Invalid amount. Must be positive.')
        return balance
    return balance + amount

def withdraw(balance, amount):
    """Return new balance after withdrawal."""
    if amount <= 0:
        print(' Invalid amount. Must be positive.')
        return balance
    if amount > balance:
        print(' Insufficient funds.')
        return balance
    return balance - amount

def get_statement(transactions):
    """Print formatted bank statement."""
    print('=' * 50)
    print(f"{'#':<4} {'TYPE':<12} {'AMOUNT':>12} {'BALANCE':>12}")
    print('=' * 50)
    for i, t in enumerate(transactions, 1):
        print(f"{'i':<4} {t['type']:<12} {t['amount']:>12,.2f} {t['balance_after']")
    print('=' * 50)

# Main program - simulated transactions for demonstration
balance = 50000.0
transactions = []

print('===== MINI BANKING SYSTEM =====')
print(f'Starting balance: {balance:,.2f} RWF')

# Simulate deposit
balance = deposit(balance, 20000)
transactions.append({'type': 'Deposit', 'amount': 20000, 'balance_after': balance})
print(f' Deposited 20,000 RWF -> New balance: {balance:,.2f} RWF')

# Simulate withdrawal
balance = withdraw(balance, 15000)
transactions.append({'type': 'Withdrawal', 'amount': 15000, 'balance_after': balance})
print(f' Withdrew 15,000 RWF -> New balance: {balance:,.2f} RWF')
```

```

# Simulate another deposit
balance = deposit(balance, 30000)
transactions.append({'type': 'Deposit', 'amount': 30000, 'balance_after': balance})
print(f' Deposited 30,000 RWF -> New balance: {balance:,.2f} RWF')

# Simulate invalid withdrawal
balance = withdraw(balance, 200000)

# Print final statement
print()
get_statement(transactions)
print(f'\n Final balance      : {balance:,.2f} RWF')
print(f' Total transactions: {len(transactions)}')

```

===== MINI BANKING SYSTEM =====

Starting balance: 50,000.00 RWF

Deposited 20,000 RWF -> New balance: 70,000.00 RWF

Withdrew 15,000 RWF -> New balance: 55,000.00 RWF

Deposited 30,000 RWF -> New balance: 85,000.00 RWF

Insufficient funds.

```

=====
#      TYPE              AMOUNT      BALANCE
=====
1   Deposit           20,000.00    70,000.00
2   Withdrawal        15,000.00    55,000.00
3   Deposit           30,000.00    85,000.00
=====

```

Final balance : 85,000.00 RWF

Total transactions: 3

Task 5: Data Processing Pipeline (5 marks)

In [5]: # Task 5: Data Processing Pipeline

```

raw_data = [
    ' Alice Uwase      | 22 | F | Kigali      | 88.5 | Computer Science ',
    ' Bob Mugisha      | 19 | M | Butare      | 72.0 | Mathematics      ',
    ' Carol Ingabire    | 24 | F | Gisenyi     | 95.5 | Data Science     ',
    ' David Habimana    | 21 | M | Kigali      | 41.0 | Computer Science ',
    ' Eve Mutoni        | 20 | F | Rwamagana   | 65.5 | Mathematics      ',
    ' Frank Nkurunz     | 23 | M | Butare      | 78.0 | Data Science     ',
    ' Grace Umutoni     | 18 | F | Kigali      | 91.0 | Computer Science ',
    ' Henri Gasabo      | 22 | M | Musanze     | 55.5 | Mathematics      ',
]

def parse_record(raw_string):
    """Parse one raw string into a clean dictionary."""
    parts = raw_string.strip().split('|')
    return {
        'name'       : parts[0].strip(),
        'age'        : int(parts[1].strip()),
        'gender'     : parts[2].strip(),
        'city'       : parts[3].strip(),
        'score'      : float(parts[4].strip()),
        'department' : parts[5].strip(),
    }

```



```

def clean_dataset(raw_list):
    """Clean all raw records into a list of dictionaries."""
    return [parse_record(r) for r in raw_list]

def filter_by_department(data, dept):
    """Return only records from a given department."""
    return [s for s in data if s['department'] == dept]

def top_performers(data, n):
    """Return top n students by score."""
    return sorted(data, key=lambda s: s['score'], reverse=True)[:n]

# Clean the dataset
dataset = clean_dataset(raw_data)

# Print all students
print('=' * 60)
print(f"{'NAME':<18} {'AGE':>4} {'G':>2} {'CITY':<12} {'SCORE':>6} {'DEPT':<18}")
print('=' * 60)
for s in dataset:
    print(f"{s['name']:<18} {s['age']:>4} {s['gender']:>2} {s['city']:<12} {s['s

# Average score per department
print('\n--- Average Score per Department ---')
departments = set(s['department'] for s in dataset)
for dept in departments:
    dept_scores = [s['score'] for s in dataset if s['department'] == dept]
    print(f" {dept:<20}: {sum(dept_scores)/len(dept_scores):.2f}")

# Top 3 performers
print('\n--- Top 3 Performers ---')
for s in top_performers(dataset, 3):
    print(f" {s['name']:<18} -> {s['score']}")

# Female students from Kigali
print('\n--- Female Students from Kigali ---')
kigali_females = [s for s in dataset if s['gender'] == 'F' and s['city'] == 'Kig
for s in kigali_females:
    print(f" {s['name']:<18} -> {s['score']}")

```

```
=====
NAME          AGE  G CITY          SCORE DEPT
=====
Alice Uwase    22  F Kigali          88.5 Computer Science
Bob Mugisha    19  M Butare          72.0 Mathematics
Carol Ingabire 24  F Gisenyi         95.5 Data Science
David Habimana 21  M Kigali          41.0 Computer Science
Eve Mutoni     20  F Rwamagana       65.5 Mathematics
Frank Nkurunz 23  M Butare          78.0 Data Science
Grace Umutoni  18  F Kigali          91.0 Computer Science
Henri Gasabo   22  M Musanze        55.5 Mathematics
```

--- Average Score per Department ---

```
Computer Science : 73.50
Data Science     : 86.75
Mathematics      : 64.33
```

--- Top 3 Performers ---

```
Carol Ingabire   -> 95.5
Grace Umutoni    -> 91.0
Alice Uwase      -> 88.5
```

--- Female Students from Kigali ---

```
Alice Uwase      -> 88.5
Grace Umutoni    -> 91.0
```